BASIC NETWORK SNIFFER USING PYTHON

Submitted By

Name: Javed Ahmed
Internship Domain: Cyber Security
Email:javedmaharofficial@gmail.com
Phone Number: +923136672617

Introduction

Cyber Security focuses on protecting computer systems, networks, and data from unauthorized access and cyber threats. Network monitoring and traffic analysis are important aspects of cyber security because they help security professionals understand how data travels across a network and identify suspicious activities.

A network sniffer is a tool used to capture and analyze network packets. It allows users to inspect network traffic and understand communication between devices. In this project, a basic network sniffer was developed using Python and the Scapy library to capture and display network packet information.

Objective

The main objectives of this project are:

- To understand the fundamentals of network packet capturing.
- To learn how network communication occurs between devices.
- To analyze packet information such as source address, destination address, and protocol type.
- To gain practical experience with Python-based network security tools.

Tools and Technologies Used

Tool	Purpose
Python 3	Programming Language
Scapy	Packet Capturing and Analysis

Tool	Purpose
Kali Linux	Development Environment
Terminal	Running Commands and Scripts

Methodology

The project was implemented using the Scapy library in Python. The sniffer continuously listens to network traffic and captures packets passing through the network interface. Whenever a packet is received, the program extracts important information and displays it on the screen.

The following information is collected from each packet:

- Source IP Address
- Destination IP Address
- Protocol Type
- Packet Length

Code Implementation

```
from scapy.all import sniff
from scapy.layers.inet import IP

def packet_callback(packet):

    if packet.haslayer(IP):

        print("\n=== Packet Captured ===")

        print("Source IP:", packet[IP].src)

        print("Destination IP:", packet[IP].dst)

        print("Protocol:", packet[IP].proto)

        print("Packet Length:", len(packet))

        print("-------------------------")

print("Starting Network Sniffer...")

sniff(prn=packet_callback, store=False)
```

Working Procedure

1. Install the Scapy library using pip.

2. Create a Python script containing the network sniffer code.
3. Execute the script with administrator/root privileges.
4. Generate network traffic by browsing websites or using ping commands.
5. Observe the captured packet information displayed by the sniffer.
6. Analyze the packet details obtained during execution.

Results

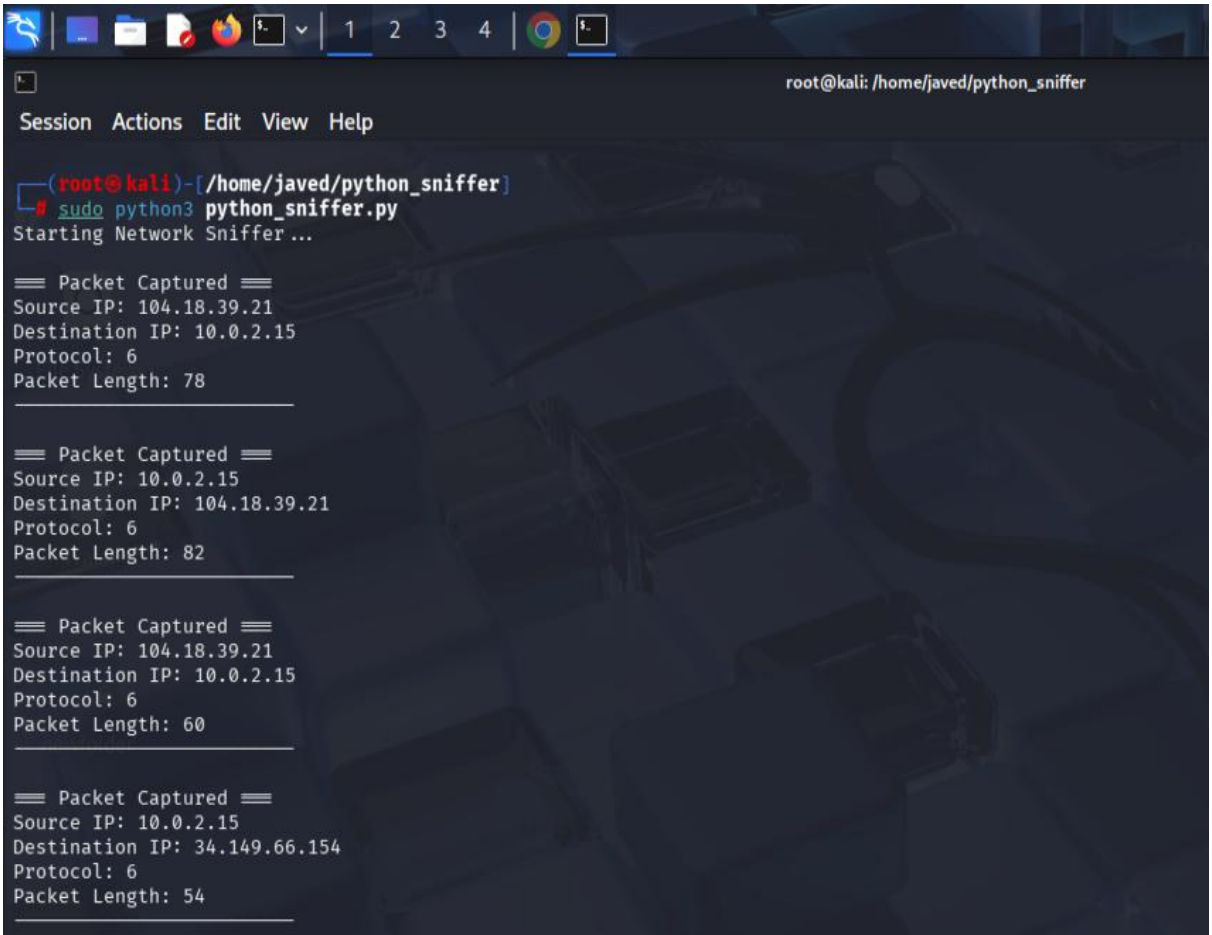
The network sniffer successfully captured live network packets and displayed their information in real time. The output included source and destination IP addresses, protocol numbers, and packet lengths.

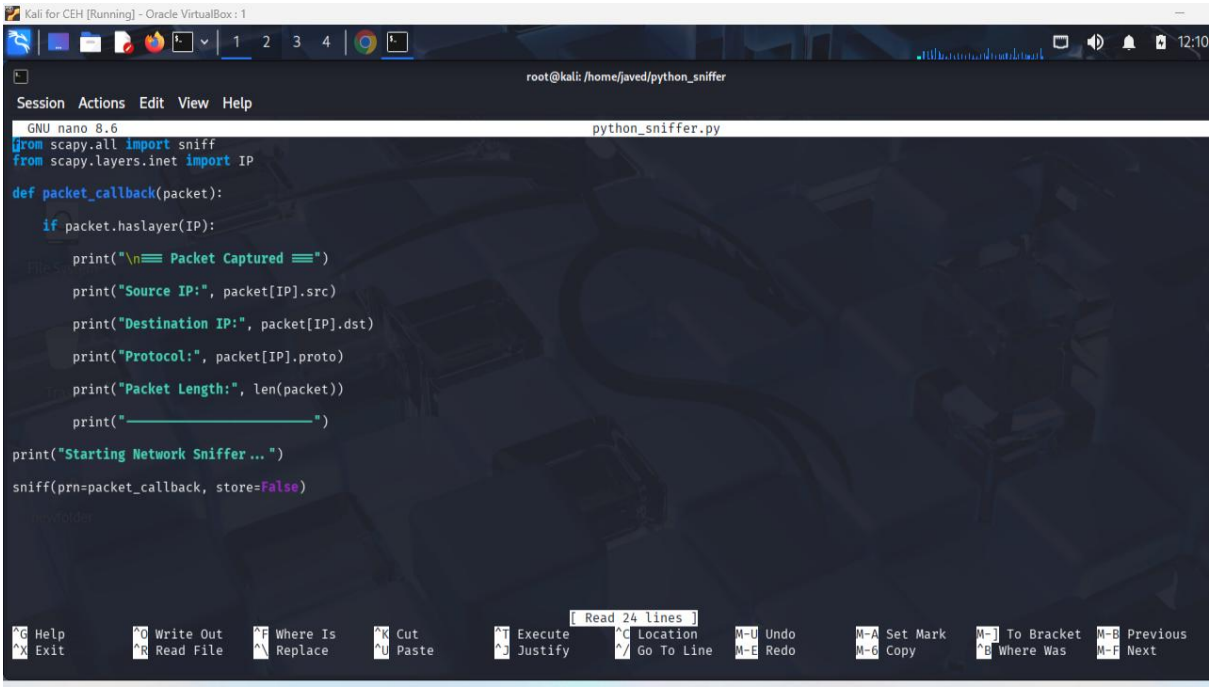
Sample Output:

```
=== Packet Captured ===
Source IP: 192.168.1.5
Destination IP: 8.8.8.8
Protocol: TCP
Packet Length: 74
--------------------------
```

The successful capture of packets demonstrates that the program can monitor and analyze network communication occurring on the system.

Screenshots





Learning Outcomes

Through this project, the following concepts were learned:

- Basics of packet sniffing.
- Network packet structure.
- IP addressing and communication.
- Protocol identification.
- Practical use of Python in cyber security.
- Traffic monitoring and analysis.

Challenges Faced

Some challenges encountered during the project included:

- Running packet capture with appropriate privileges.
- Understanding packet structures and protocols.
- Installing and configuring required Python libraries.

These challenges were resolved through research and testing in the Kali Linux environment.

Conclusion

This project successfully demonstrated the implementation of a basic network sniffer using Python and Scapy. The developed tool was able to capture and analyze network traffic in real time. The project enhanced understanding of packet structures, network communication, and traffic monitoring techniques that are widely used in cyber security operations. The knowledge gained from this task provides a strong foundation for advanced topics such as intrusion detection, packet analysis, and network forensics.

Task #02

KEY LOGGING SIMULATION USING PYTHON

Introduction

Key logging is the process of recording keyboard inputs made by a user on a computer system. While keyloggers are often associated with malicious activities such as credential theft and unauthorized surveillance, they are also studied in cybersecurity education to understand how attackers collect sensitive information and how security solutions detect such behavior.

In this project, a basic key logger was developed using Python in a controlled and authorized environment. The program captures keyboard inputs, stores them in a local log file, and records session start and end times. This project demonstrates the functionality of keystroke monitoring while emphasizing ethical and defensive cybersecurity practices.

Objective

The objectives of this project are:

- To understand how keyboard events can be captured programmatically.
- To learn the working mechanism of keyloggers.
- To study the security risks associated with keystroke logging.
- To gain practical experience with Python event-handling libraries.

- To understand how cybersecurity professionals analyze and defend against keylogging attacks.

Tools and Technologies Used

Tool	Purpose
Python 3	Programming Language
Pynput Library	Keyboard Event Monitoring
Windows PowerShell	Program Execution
Notepad	Viewing Logged Data

Methodology

The key logger was implemented using Python's Pynput library, which allows monitoring of keyboard events.

The program performs the following actions:

1. Starts a logging session and records the start time.
2. Monitors all keyboard inputs.
3. Stores typed characters in a text file.
4. Records special keys such as Enter, Tab, Backspace, Shift, and Ctrl.
5. Stops logging when the ESC key is pressed.
6. Records the session end time.

The captured keystrokes are stored locally in a log file named:

```
keylog.txt
```

Code Implementation

The implementation was developed using Python and consists of the following major components:

Session Management

The program records the start and end time of each logging session.

Key Capture

Printable characters are stored directly in the log file.

Special Key Detection

Special keys are identified and recorded using descriptive labels such as:

```
[ENTER]
[TAB]
[BACKSPACE]
[SHIFT]
[CTRL]
```

Session Termination

Pressing the ESC key safely stops the program and closes the logging session.

Working Procedure

1. Install the required Python library:

```
pip install pynput
```

2. Create the Python script.
3. Execute the program:

```
python KeyLogger.py
```

4. Type sample text in any application.
5. Press ESC to stop logging.
6. Open the generated log file to review captured keystrokes.

Results

The key logger successfully captured keyboard input and stored it in a local text file.

The generated log file contained:

- Session start and end timestamps.
- User-entered text.
- Special key events.
- Multiple logging sessions.

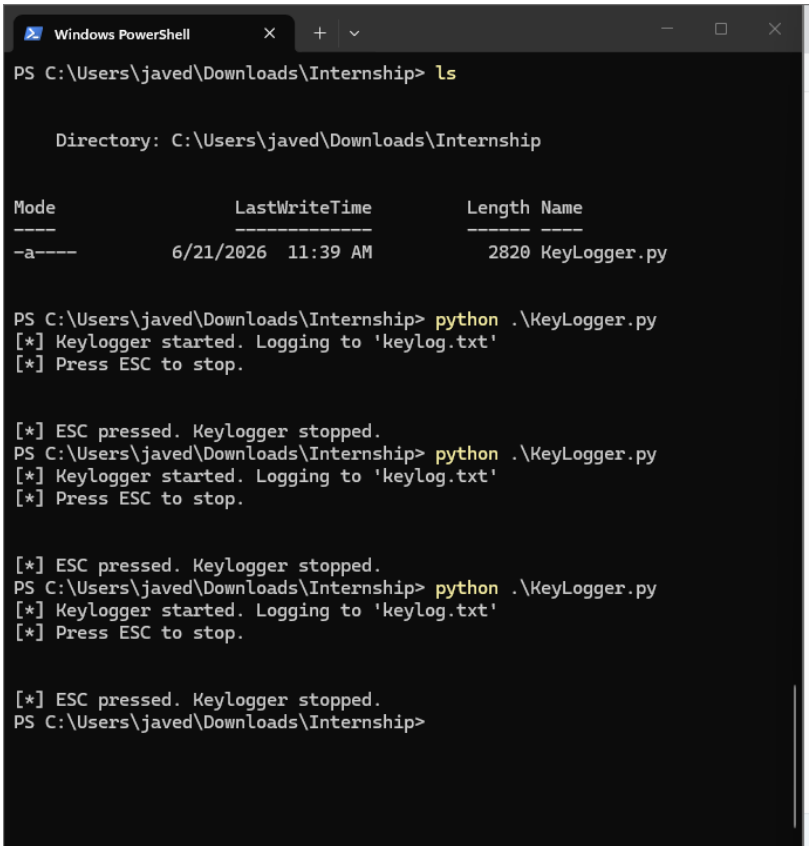
Sample output:

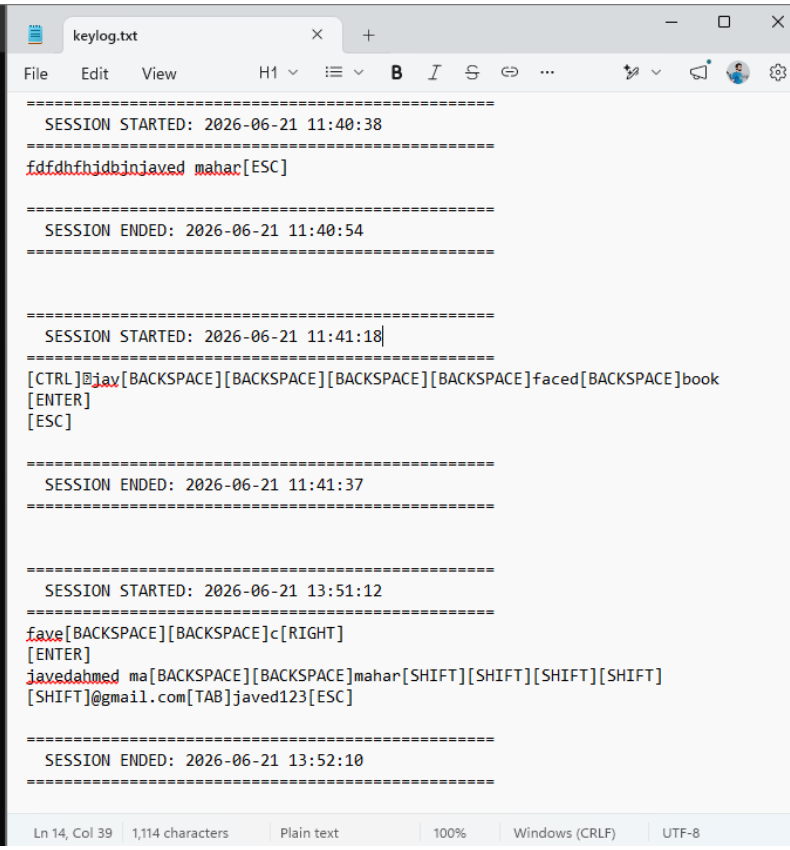
```
================================================
SESSION STARTED: 2026-06-21 13:51:12
================================================
```

```
javedahmed_mahar
javed123

[ESC]

================================================
SESSION ENDED: 2026-06-21 13:52:10
================================================
```

The successful logging of both printable and special keys demonstrates that the application accurately monitored keyboard events in real time.

Screenshots





Security Risks of Keyloggers

Keyloggers pose significant security risks because they can capture:

- Usernames
- Passwords
- Banking credentials
- Personal messages
- Confidential information

Attackers may use keyloggers to steal sensitive information without the user's knowledge. Therefore, understanding their operation helps cybersecurity professionals develop effective detection and prevention mechanisms.

Mitigation Techniques

The following measures can help protect systems against keyloggers:

- Install reputable antivirus software.
- Keep operating systems updated.
- Use endpoint detection and response (EDR) solutions.
- Enable multi-factor authentication (MFA).
- Avoid downloading software from untrusted sources.
- Monitor system processes and startup applications.

Learning Outcomes

Through this project, the following concepts were learned:

- Keyboard event handling in Python.
- Real-time keystroke monitoring.
- File handling and logging mechanisms.
- Ethical considerations in cybersecurity.
- Security risks associated with keylogging.
- Defensive strategies against information theft.

Ethical Considerations

This project was conducted solely for educational purposes within a controlled and authorized environment. No unauthorized monitoring, credential theft, persistence mechanisms, or remote transmission of captured data were implemented. The objective was to understand the risks associated with keylogging and promote awareness of cybersecurity best practices.

Challenges Faced

During implementation, the following challenges were encountered:

- Understanding keyboard event listeners.
- Handling special key events correctly.
- Managing session start and end timestamps.
- Ensuring proper file writing and logging.

These challenges were resolved through testing and debugging of the Python application.

Conclusion

This project successfully demonstrated the implementation of a basic key logger using Python and the Pynput library. The program was able to capture keyboard events, record special keys, and maintain detailed session logs. The project enhanced understanding of keystroke monitoring techniques and highlighted the security risks posed by keyloggers. Additionally, it provided practical experience in Python programming and reinforced the importance of ethical cybersecurity practices.
